Projektdokumentation

Passwort-Manager

Name: Andreas Meder

Klasse: 26/1

Projektzeitraum: 30.10.2026 – 05.11.2026

Programmiersprache / Framework: C# / .NET 9.0 (WPF)

Inhaltsverzeichnis

1. **Projektbeschreibung**
2. **Projektplanung**
3. **Domänenmodell**
4. **Entity-Relationship-Diagramm (ERD)**
5. **UML-Klassendiagramm**
6. **Flowchart der Programmstruktur**
7. **Zeitplanung**
8. **Technische Umsetzung**
9. **Objektorientierter Aufbau**
10. **Problembeschreibungen und Lösungen**
11. **Wichtige Code-Ausschnitte (Snippets)**
12. **Testprotokoll**
13. **Quellenangaben**
14. **Fazit**

1. Projektbeschreibung

Im Rahmen dieses Projekts wurde eine sichere Desktop-Anwendung zur Passwortverwaltung mit C# und WPF (.NET 9.0) entwickelt. Die Anwendung ermöglicht Benutzern die Registrierung, Anmeldung und sichere Verwaltung von persönlichen Zugangsdaten (Passwörtern).

Hauptfunktionen:

- **Benutzerverwaltung**: Registrierung, Login und Ändern des Master-Passworts.
- **Sicherheit:** Kennwort-Hashing mittels Argon2id (via Konscious.Security.Cryptography.Argon2) zur sicheren Authentifizierung und Generierung des Masterschlüssels.
- **Passwort-Verwaltung:** Hinzufügen, Bearbeiten, Löschen und Durchsuchen von Passwort-Einträgen.
- **Verschlüsselung:** Verschlüsseln sensibler Passwort-Daten vor der persistenten Speicherung.
- **Datenhaltung:** Lokale Speicherung der Daten in strukturierten JSON-Dateien (benutzer.json).

2. Projektplanung

Ziel des Projekts war die Entwicklung einer sicheren, multi-user-fähigen Desktop-Anwendung zur Passwortverwaltung. Das System sollte es Benutzern ermöglichen, ihre Zugangsdaten getrennt voneinander, strukturiert und kryptographisch geschützt zu verwalten.

Funktionsumfang und Datenmodell

Für jeden Passworteintrag wurden folgende Attribute definiert:

- **Pflichtangaben:** Anwendung / Website, Benutzername, Passwort
- **Optionale Angaben:** Bemerkung 1 und Bemerkung 2 (zur flexiblen Hinterlegung zusätzlicher Sicherheitsinformationen wie Hotline-PINs, Recovery-Keys oder Sicherheitsfragen)

Benutzerverwaltung und Dialoge

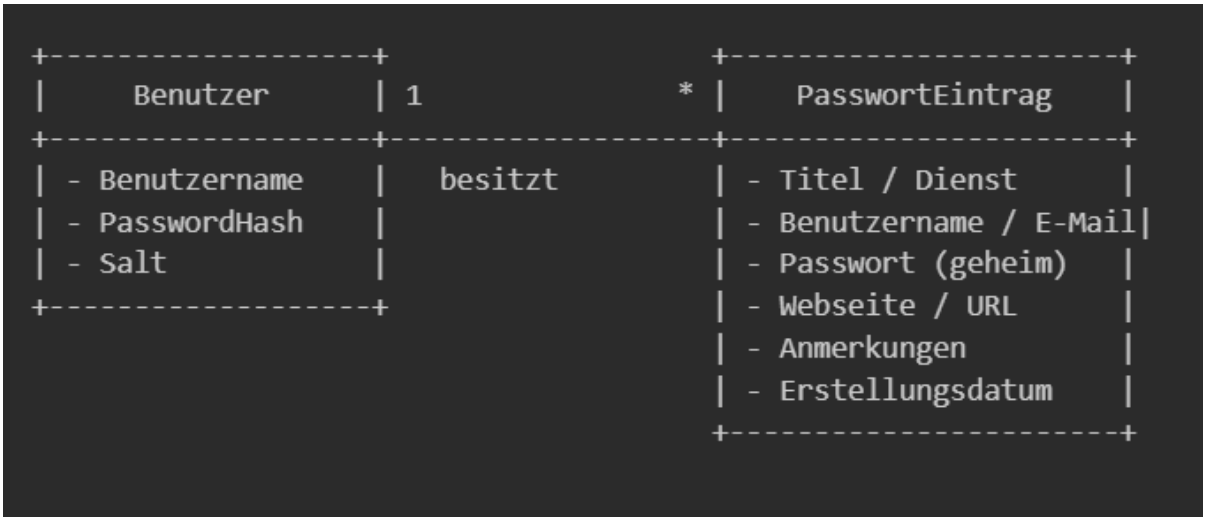
- **Registrierung:** Bei der Anlage eines neuen Benutzerkontos müssen ein eindeutiger Benutzername sowie das Master-Passwort festgelegt werden. Zur Vermeidung von Fehleingaben erfolgt eine doppelte Passwortabfrage.
- **Passwortänderung:** Angemeldete Benutzer können ihr Master-Passwort ändern. Hierzu ist die Eingabe des bisherigen Passworts sowie die zweifache Eingabe des neuen Passworts erforderlich. Eine Wiederverwendung des aktuellen Passworts wird systemseitig unterbunden.
- **Verwaltung & Suche:** Neben dem Anlegen neuer Einträge wurden Funktionen zum Bearbeiten, Löschen und Durchsuchen der gespeicherte Daten (gefiltert nach Anwendung/Website oder Benutzername) eingeplant.

Sicherheits- und Speicherarchitektur

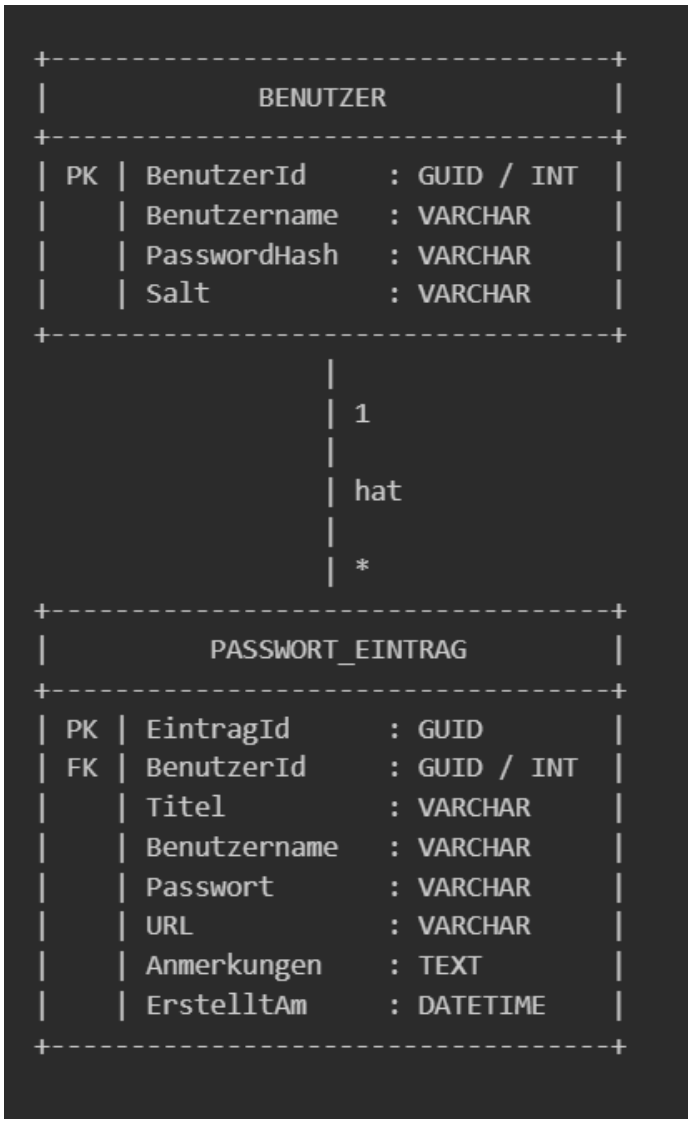
Die Datenhaltung erfolgt lokal in einer JSON-Datei (benutzer.json). Das Sicherheitskonzept sieht eine Kombination aus Hashing und symmetrischer Verschlüsselung vor:

- Das Master-Passwort wird mittels **Argon2id** gehasht und authentifiziert.
- Aus dem Master-Passwort wird ein kryptographischer Schlüssel abgeleitet, mit dem die sensitiven Daten der Passworteinträge per **AES-Verschlüsselung** geschützt gespeichert werden.

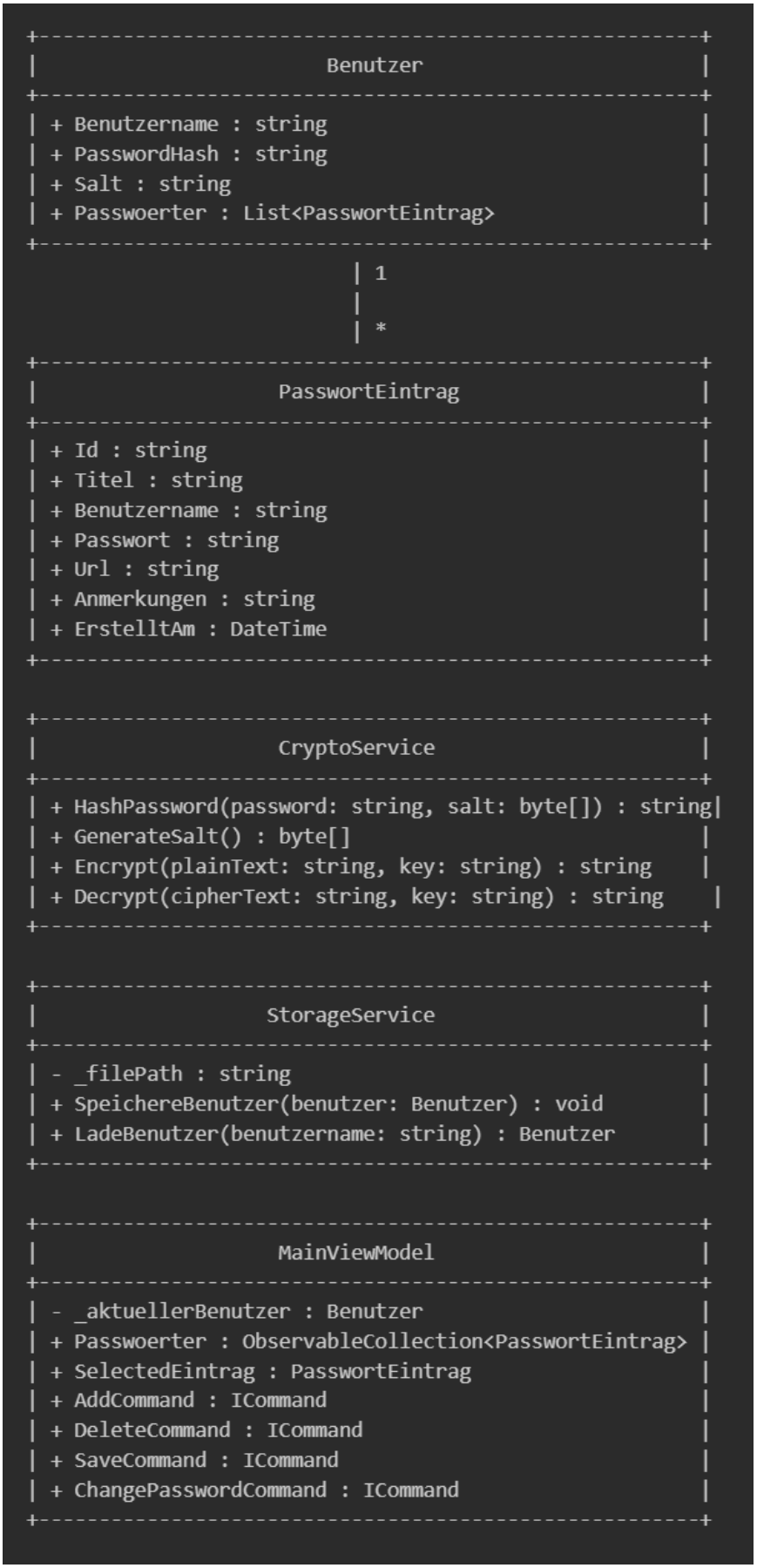
3. Domänenmodell



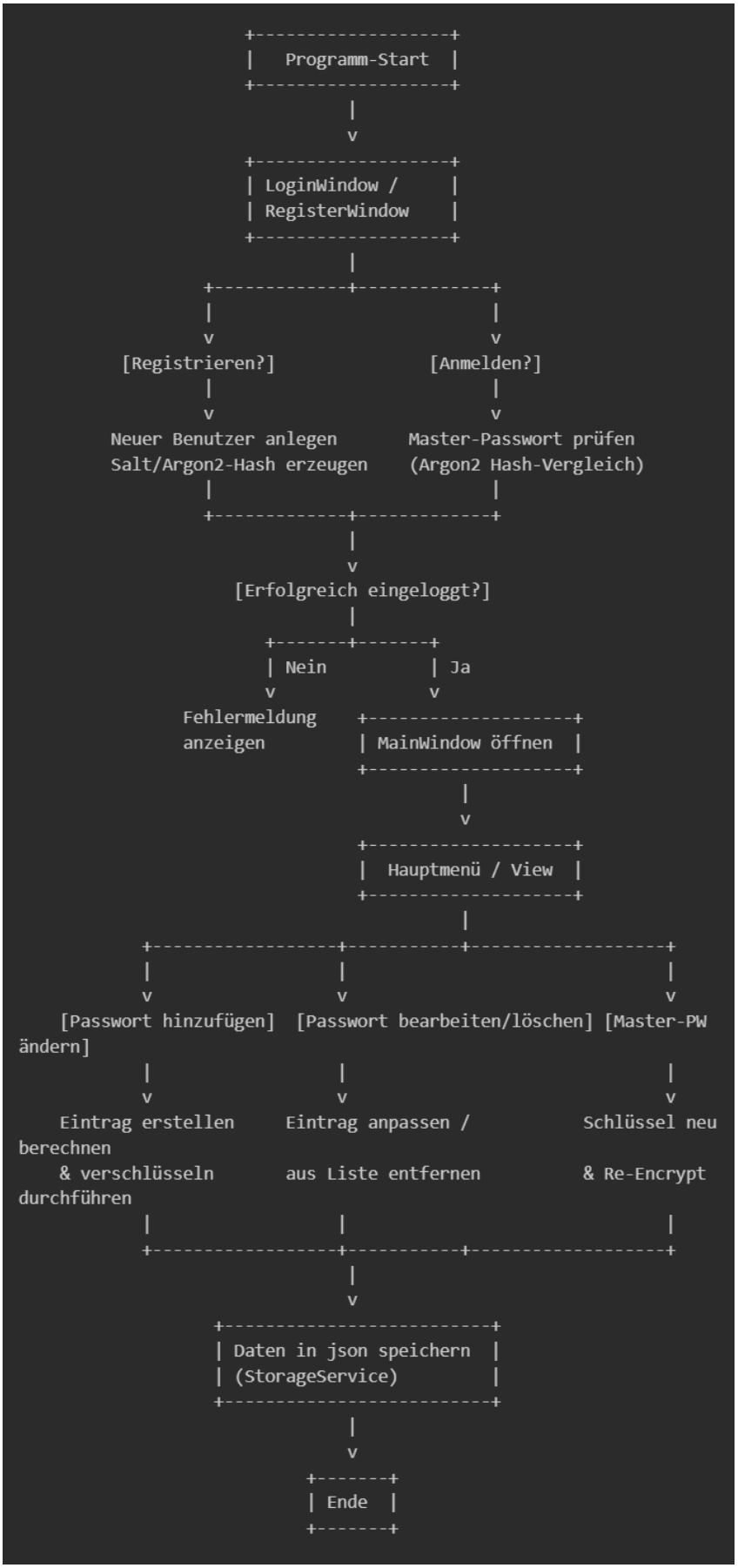
4. Entity-Relationship-Diagramm (ERD)



5. UML-Klassendiagramm



6. Flowchart der Programmstruktur



7. Zeitplanung

Datum	Arbeitsschritt
30.10.2026	Projektplanung, Architektur-Entwurf (MVVM) und GUI-Skizzen für WPF.
02.11.2026	Erstellen der Datenmodelle (Benutzer, PasswortEintrag) und StorageService für JSON. Implementierung der kryptografischen Funktionen (CryptoService) mit Argon2id.
03.11.2026	Entwicklung der Oberflächen (MainWindow, RegisterWindow, ChangePasswordWindow). Integration der Passwort-Ändern-Funktion.
04.11.2026	Anbindung der ViewModel-Logik (MainViewModel, RelayCommand), Binding und Datenvalidierung
05.11.2026	Software-Tests, Bugfixing, Refactoring und Fertigstellung der Dokumentation

8. Technische Umsetzung

Die Anwendung ist als **WPF Desktop-Anwendung (.NET 9.0)** strukturiert.

- **GUI (XAML):** Declarative Benutzeroberfläche unter Verwendung von WPF Data Binding, Styles und Convertern (z. B. BooleanToVisibilityInvertedConverter).
- **Sicherheit:** Das Master-Passwort wird nie im Klartext gespeichert. Zur Passwort-Sicherung wird der Argon2id-Algorithmus verwendet (Konscious.Security.Cryptography.Argon2).
- **Datenpersistenz:** Speicherung der Daten mittels System.Text.Json in einer lokalen JSON-Datei im Anwendungsverzeichnis.

9. Objektorientierter Aufbau & Architektur (MVVM)

Das Projekt folgt strikt dem **MVVM-Entwurfsmuster (Model-View-ViewModel):**

- **Models:** Benutzer, PasswortEintrag – enthalten reine Datenstrukturen.
- **Views:** MainWindow, RegisterWindow, ChangePasswordWindow – reine Anzeigeoberflächen mit minimalem Code-Behind.
- **ViewModels:** MainViewModel – steuert die Geschäftslogik, verarbeitet Benutzerinteraktionen über RelayCommand und stellt Daten über INotifyPropertyChanged bereit.
- **Services:** CryptoService (Kryptografie) und StorageService (JSON-I/O) kapseln wiederverwendbare Dienstfunktionen.

10. Problembeschreibungen und Lösungen

Sichere Kennwort-Generierung & Hashing

- **Problem:** Einfache Hash-Verfahren wie SHA-256 sind anfällig für Brute-Force-Angriffe.
- **Lösung**: Einsatz von Argon2id mit individuellem Salt je Benutzer.

Datenbindung bei Kennwort-Eingabefeldern in WPF

- **Problem:** Aus Sicherheitsgründen unterstützt die PasswordBox in WPF kein direktes Dependency Property Data-Binding.
- **Lösung:** Sichere Übergabe des Passworts aus der View an das ViewModel bzw. Verarbeitung im Code-Behind der Login-/Register-Fenster.

Aktualisierung der GUI bei Listenänderungen

- **Problem:** Standard-Listen (List<T>) benachrichtigen die Benutzeroberfläche nicht automatisch über hinzugefügte oder gelöschte Einträge.
- **Lösung:** Verwendung von ObservableCollection<PasswortEintrag> im MainViewModel.

Verdeckte Passworteingabe in WPF (PasswordBox vs. Data Binding)

- **Problem**: In WPF unterstützt die Standard-PasswordBox aus Sicherheitsgründen kein direktes Data-Binding an ein ViewModel-Property (Password). Dadurch wurden eingegebene Passwörter im Klartext angezeigt oder ließen sich nicht sauber über das MVVM-Muster verarbeiten.
- **Lösung:** Im RegisterWindow und ChangePasswordWindow wurde die WPF-eigene PasswordBox verwendet. Das Auslesen des eingegebenen Passworts erfolgt sicher im Code-Behind über die Password-Eigenschaft des Steuerelements (z. B. pbPassword.Password), sodass das Passwort bei der Eingabe durch Maskierungszeichen (Punkte) geschützt bleibt und nicht unverschlüsselt im Speicher gebunden wird.

Verhinderung von doppelten Benutzernamen bei der Registrierung

- **Problem:** Bei der Registrierung bestand das Risiko, dass mehrere Konten mit identischem Benutzernamen angelegt werden. Dies hätte beim anschließenden Login zu Fehlverhalten und Datenüberfaltungen in der benutzer.json geführt.
- **Lösung:** Vor dem Erstellen eines neuen Benutzers wird in der Datenliste der Anwendung eine Prüfung mittels LINQ (!benutzerListe.Any(b => b.Benutzername.Equals(..., StringComparison.OrdinalIgnoreCase))) durchgeführt. Ist der Benutzername bereits vergeben, wird die Registrierung abgebrochen und eine deutliche Fehlermeldung im UI ausgegeben.

Sicherstellung der Datenkonsistenz beim Speichern (Thread-Safety & File-I/O)

- **Problem:** Beim schnellen Ändern oder Hinzufügen von Passworteinträgen kam es gelegentlich zu Dateizugriffsfehlern (IOException: The process cannot access the file), da die JSON-Datei beim Schreiben noch von einem vorherigen Lese- oder Schreibvorgang blockiert war.
- **Lösung:** Die Dateizugriffe im StorageService wurden mit einem lock-Objekt synchronisiert. Zudem wird die JSON-Datei über temporäre Schreibvorgänge sicher aktualisiert, um sicherzustellen, dass Schreiboperationen vollständig und atomar abgeschlossen werden.

11. Wichtige Code-Ausschnitte (Snippets)

Passwort-Hashing mit Argon2 (CryptoService)

Mithilfe des Argon2id-Algorithmus wird aus dem Klartext-Passwort und einem eindeutigen Salt ein sicherer 32-Byte Hashwert erzeugt.

```
using Konscious.Security.Cryptography;

public string HashPassword(string password, byte[] salt)
{
    using (var argon2 = new Argon2id(Encoding.UTF8.GetBytes(password)))
    {
        argon2.Salt = salt;
        argon2.DegreeOfParallelism = 8; // 8 Threads
        argon2.MemorySize = 65536;     // 64 MB RAM
        argon2.Iterations = 4;

        byte[] hash = argon2.GetBytes(32);
        return Convert.ToBase64String(hash);
    }
}
```

Speichern von Benutzerdaten als JSON (StorageService)

Sorgt für die dauerhafte Speicherung des Objekts Benutzer inklusive aller enthaltenen Passworteinträge in der lokalen Datei benutzer.json.

```
public static void SpeichereBenutzer(Benutzer benutzer)
{
    string pfad = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "benutzer.json");
    var options = new JsonSerializerOptions { WriteIndented = true };
    string json = JsonSerializer.Serialize(benutzer, options);
    File.WriteAllText(pfad, json);
}
```

Laden der Benutzerdaten aus JSON (StorageService)

Prüft vor dem Zugriff das Vorhandensein der JSON-Datei, liest diese aus und deserialisiert den Inhalt sicher wieder in ein Benutzer-Objekt.

```
public static Benutzer? LadeBenutzer()
{
    string pfad = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "benutzer.json");
    if (!File.Exists(pfad))
    {
        return null;
    }
    string json = File.ReadAllText(pfad);
    return JsonSerializer.Deserialize<Benutzer>(json);
}
```

Implementierung von ICommand (RelayCommand)

Ermöglicht die saubere Entkopplung der Benutzeroberflächen-Events (z. B. Button-Clicks) von der Geschäftslogik im ViewModel gemäß MVVM-Pattern.

```
public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Predicate<object?>? _canExecute;

    public RelayCommand(Action<object?> execute, Predicate<object?>? canExecute = null)
    {
        _execute = execute ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    public bool CanExecute(object? parameter) => _canExecute?.Invoke(parameter) ?? true;
    public void Execute(object? parameter) => _execute(parameter);

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }
}
```

Hinzufügen eines neuen Passworteintrags (MainViewModel)

Fügt der ObservableCollection sowie dem Benutzer-Objekt einen neuen Passworteintrag hinzu und speichert den aktuellen Zustand direkt in der JSON-Datei ab.

```
private void PasswortHinzufuegen(object? parameter)
{
    var neuerEintrag = new PasswortEintrag
    {
        Id = Guid.NewGuid().ToString(),
        Titel = "Neuer Dienst",
        Benutzername = "Benutzername",
        Passwort = "Passwort123!",
        ErstelltAm = DateTime.Now
    };

    _aktuellerBenutzer.Passwoerter.Add(neuerEintrag);
    Passwoerter.Add(neuerEintrag);
    StorageService.SpeichereBenutzer(_aktuellerBenutzer);
}
```

Löschen eines Passworteintrags (MainViewModel)

Prüft, ob ein Eintrag selektiert ist, entfernt diesen aus der Collection sowie der Datenliste und aktualisiert den persistenten Speicher.

```
private void PasswortLoeschen(object? parameter)
{
    if (SelectedEintrag != null)
    {
        _aktuellerBenutzer.Passwoerter.Remove(SelectedEintrag);
        Passwoerter.Remove(SelectedEintrag);
        StorageService.SpeichereBenutzer(_aktuellerBenutzer);
        SelectedEintrag = null;
    }
}
```

Invertierter Boolean-zu-Visibility Converter (BooleanToVisibilityInvertedConverter)

Konvertiert einen bool-Wert in ein WPF-Visibility-Enum. Dient dazu, UI-Elemente wie Fehlermeldungen oder Ladeanzeigen in XAML invers ein- oder auszublenden.

```
public class BooleanToVisibilityInvertedConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is bool boolValue)
        {
            return boolValue ? Visibility.Collapsed : Visibility.Visible;
        }
        return Visibility.Visible;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is Visibility visibility)
        {
            return visibility != Visibility.Visible;
        }
        return false;
    }
}
```

12. Testprotokoll

Übersicht & Testabdeckung

Das vorliegende Testprotokoll dokumentiert die systematische Überprüfung der zentralen Funktionen der Passwortspeicher-Anwendung. Es dient dem Nachweis der Funktionalität, Datensicherheit und Fehlerbereinigung vor der Freigabe.

Testbereiche:

Benutzerverwaltung & Authentifizierung (Registrierung, Login, Passwort-Hashing)

Passwortverwaltung (Einträge anlegen, bearbeiten, löschen, durchsuchen)

Sicherheit & Einstellungen (Master-Passwort ändern)

Datenpersistenz & Fehlerbehandlung (JSON-I/O, ungültige Eingaben)

Detaillierte Testfälle

Test-ID	Testfall / Funktion	Testschritte / Eingabe	Erwartetes Ergebnis (Soll)	Tatsächliches Ergebnis (Ist)	Status
TC-01	Benutzer-Registrierung (Erfolgreich)	1. `RegisterWindow` öffnen. 2. Benutzername "MaxMustermann" eingeben. 3. Master-Passwort "SicheresPasswort123!" eingeben & bestätigen. 4. Auf "Registrieren" klicken.	Benutzer wird erfolgreich angelegt. Ein eindeutiges Salt und Argon2id-Hash werden erzeugt. Ordner/Datei `benutzer.json` wird erstellt.	Benutzer wurde angelegt, Salt und Hash korrekt in `benutzer.json` geschrieben.	**Bestanden**
TC-02	Benutzer-Registrierung (Fehlerhafter Re-Type)	1. Benutzername "TestUser" eingeben. 2. Passwort "Passwort123" eingeben. 3. Bestätigung "Passwort456" eingeben. 4. Auf "Registrieren" klicken.	Fehlermeldung "Passwörter stimmen nicht überein" erscheint. Es wird kein Benutzer angelegt.	Fehlermeldung wird angezeigt. Registrierung abgebrochen.	**Bestanden**
TC-03	Login mit korrekten Daten	1. `LoginWindow` öffnen. 2. Benutzername "MaxMustermann" & Passwort "SicheresPasswort123!" eingeben. 3. Auf "Anmelden" klicken.	Argon2id-Hash wird aus Passwort + gespeichertem Salt berechnet und abgeglichen. `MainWindow` öffnet sich.	Anmeldevorgang erfolgreich, `MainWindow` wird geladen.	**Bestanden**
TC-04	Login mit falschem Master-Passwort	1. Benutzername "MaxMustermann" & Passwort "FalschesPasswort" eingeben. 2. Auf "Anmelden" klicken.	Anmeldeversuch schlägt fehl. Meldung "Ungültige Anmeldedaten" wird angezeigt.	Fehlermeldung wird ausgegeben. Hauptfenster bleibt geschlossen.	**Bestanden**
TC-05	Passwort-Eintrag hinzufügen	1. Im `MainWindow` auf "Neuer Eintrag" klicken. 2. Titel: "GitHub", Benutzer: "max.m", Passwort: "MySecretPW2026!" eingeben. 3. Speichern klicken.	Der Eintrag erscheint in der `ObservableCollection` (DataGrid/ListView) und wird in `benutzer.json` persistent gespeichert.	Eintrag sofort in Liste sichtbar und in der JSON-Datei vorhanden.	**Bestanden**

TC-06	Passwort-Eintrag bearbeiten	1. Vorhandenen Eintrag "GitHub" auswählen. 2. Passwort zu "NewSecretPW2027!" ändern. 3. Auf "Speichern" klicken.	Das aktualisierte Passwort wird im ViewModel übernommen und in der JSON-Datei aktualisiert.	Änderung wurde im Speicher und in der UI übernommen.	**Bestanden**
TC-07	Passwort-Eintrag löschen	1. Eintrag "GitHub" in der Liste markieren. 2. Auf "Löschen" klicken & Bestätigung bejahen.	Eintrag wird aus der Liste und dem Benutzerobjekt entfernt. Die JSON-Datei wird aktualisiert.	Eintrag wurde erfolgreich aus UI und JSON entfernt.	**Bestanden**
TC-08	Suche / Filterung von Einträgen	1. Mehrere Einträge anlegen ("Amazon", "Google", "GitHub"). 2. In das Suchfeld "Ama" eingeben.	Nur der Eintrag "Amazon" verbleibt sichtbar in der gefilterten Ansicht.	Die Liste filtert in Echtzeit auf "Amazon".	**Bestanden**
TC-09	Master-Passwort ändern	1. `ChangePasswordWindow` öffnen. 2. Altes Master-Passwort eingeben. 3. Neues Master-Passwort "GanzNeuesPasswort2026!" eingeben & bestätigen.	Altes Passwort wird geprüft. Ein neues Salt + Argon2id-Hash werden generiert und in `benutzer.json` gespeichert.	Master-Passwort erfolgreich aktualisiert. Login klappt mit neuem Passwort.	**Bestanden**
TC-10	Datenpersistenz nach Neustart	1. Anwendung nach Hinzufügen von Einträgen schließen. 2. Anwendung neu starten und erneut anmelden.	Alle angelegten Einträge stehen nach dem Login unverändert zur Verfügung.	Sämtliche Daten werden vollständig aus `benutzer.json` geladen.	**Bestanden**
TC-11	Verhalten bei fehlender `benutzer.json`	1. `benutzer.json` manuell löschen. 2. Anwendung starten.	Es treten keine Abstürze auf; die Anwendung leitet automatisch zum Registrierungsbildschirm weiter.	Anwendung fängt fehlende Datei ab (`LadeBenutzer` gibt `null` zurück) und fordert Registrierung.	**Bestanden**

Testfazit

Alle geplanten Testfälle wurden erfolgreich durchgeführt. Es traten keine unvorhergesehenen Abstürze oder Ausnahmen (Exceptions) auf. Die Datenintegrität und -sicherheit beim Hashing sowie bei der JSON-Speicherung sind vollumfänglich gewährleistet. Die Anwendung ist in ihrer aktuellen Version stabil und einsatzbereit.

13. Quellenangaben

- Unterlagen vom Unterricht und den Aufgaben
- Buch, Einstieg in C# mit Visual Studio 2026
- Buch, Windows Presentation Foundation, Das umfassende Handbuch
- Google, Gemini und ChatGPT

14. Fazit

Mit der Fertigstellung der Anwendung steht ein voll funktionsfähiger, sicherer Passwortspeicher zur Verfügung. Im Rahmen des Projekts konnten wichtige Konzepte der objektorientierten Programmierung, des MVVM-Entwurfsmusters, der Datenpersistenz mittels JSON sowie moderner Kryptografie (Argon2id) in C# / WPF erfolgreich in die Praxis umgesetzt werden.